

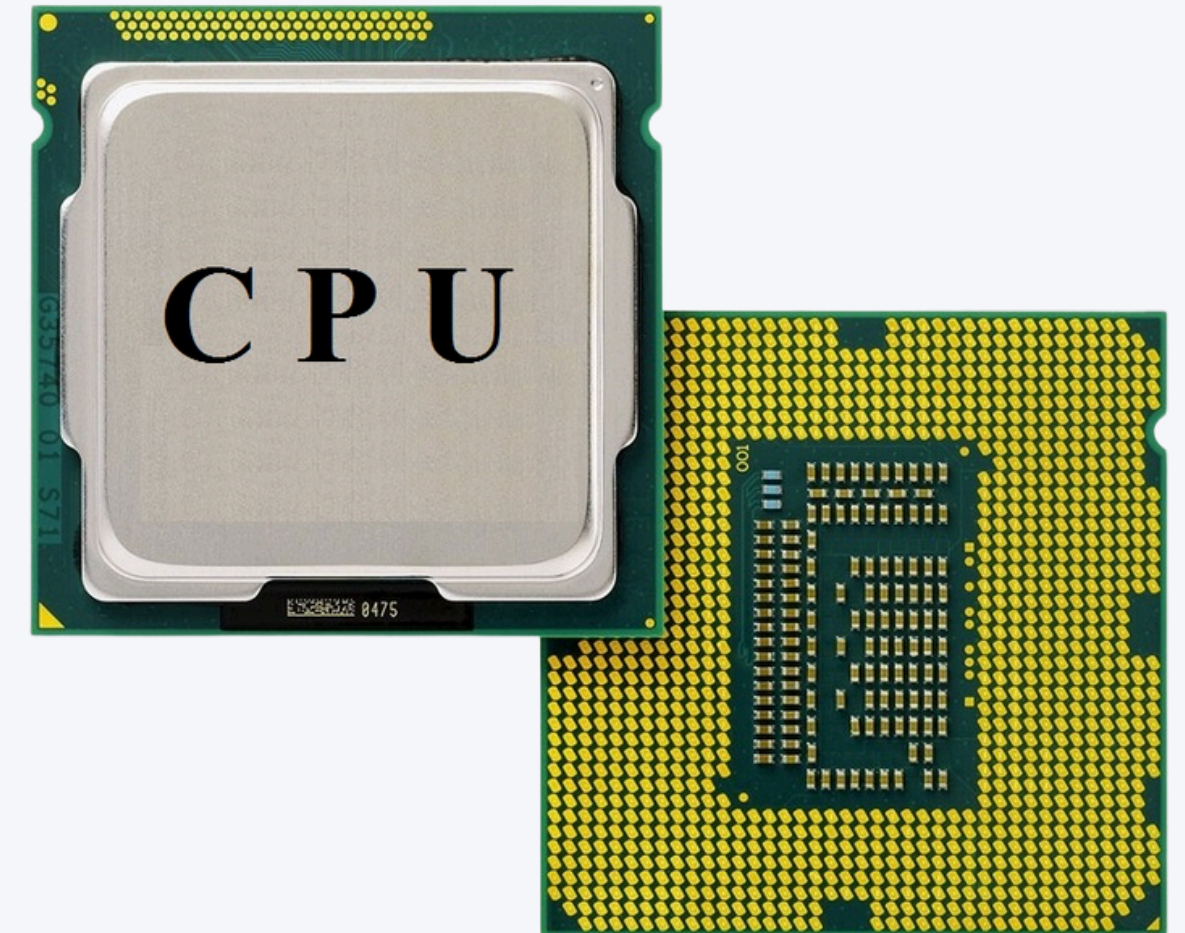
Introduction To GPU Acceleration Basics

Understanding CPU vs. GPU Architecture

The Central Processing Unit (CPU)

Key Characteristics:

- Few powerful cores (typically 4-64)
- Optimized for sequential processing
- Large cache memory
- Complex control units for branch prediction
- Designed for low-latency operations
- General-purpose processing



Introduction To GPU Acceleration Basics

Understanding CPU vs. GPU Architecture

The Graphics Processing Unit (GPU)

Key Characteristics:

- Many simple cores (thousands)
- Optimized for parallel processing
- Smaller cache per core
- Simpler control units
- Designed for high-throughput operations
- Specialized for mathematical operations



Introduction To GPU Acceleration Basics



CPU vs. GPU: Core Architecture Comparison

Aspect	CPU	GPU
Cores	Few powerful cores (4-64)	Many simple cores (thousands)
Cache	Large (MB per core)	Small (KB per core)
Control Logic	Complex (branch prediction, out-of-order execution)	Simple
Optimization	Low latency	High throughput
Memory Access	Optimized for random access	Optimized for sequential access

When to Use CPU vs. GPU

CPU Excels At:

- Tasks requiring complex decision-making
- Sequential processes with many branches
- Operations on small datasets that fit in cache
- Tasks with unpredictable memory access patterns
- Single-threaded applications

GPU Excels At:

- Highly parallel computations
- Mathematically intensive operations
- Working with large datasets
- Regular, predictable memory access patterns
- SIMD (Single Instruction, Multiple Data) operations

The Evolution of GPU Computing

1990s - Early GPUs:

- Dedicated hardware for rendering graphics
- Fixed function pipeline
- Limited to graphics applications

Early 2000s - Programmable Shaders:

- Introduction of programmable elements
- Still primarily for graphics

2006-2007 - GPGPU Emerges:

- General-Purpose computing on GPUs
- NVIDIA introduces CUDA
- AMD introduces Stream (later OpenCL)

2010s - AI Renaissance:

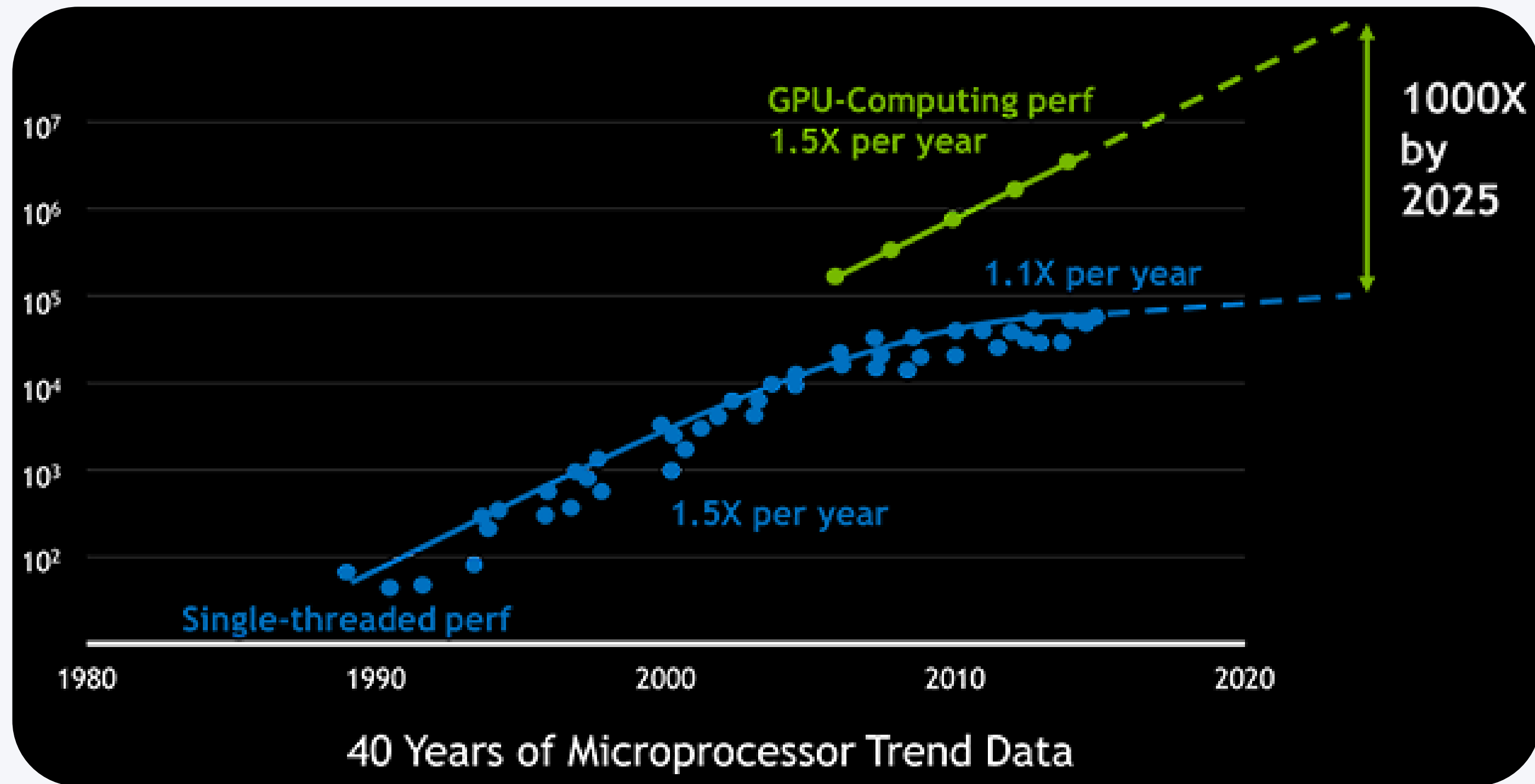
- GPUs become essential for deep learning
- NVIDIA's cuDNN library
- Specialized hardware (Tensor Cores)

2020s - Specialized Computing:

- AI-optimized GPU architectures
- Multi-GPU systems
- GPU acceleration in cloud computing

Introduction To GPU Acceleration Basics

GPU Computing Performance Growth



- CPU performance growth slowed to ~1.1x per year
- GPU performance continues to grow at ~1.5x per year
- By 2025, GPUs projected to be 1000x faster than when first introduced for computing

Introduction To GPU Acceleration Basics

Modern GPU Architecture

Key Components:

- Streaming Multiprocessors (SMs): The basic processing blocks
- CUDA Cores: Individual processing units within SMs
- Tensor Cores: Specialized for matrix multiplication (AI workloads)

Memory Hierarchy:

Global Memory (VRAM):

- Main large storage accessible by all components
- Highest capacity but slowest access

Shared Memory:

- Fast memory shared between threads in the same block
- Enables thread communication and serves as programmable cache

L1/L2 Cache:

- Automatic buffer storage that reduces memory access times
- L1 is private to each SM; L2 is shared across the GPU

Registers:

- Ultra-fast storage for individual thread variables
- Fastest but very limited capacity; private to each thread

Introduction To GPU Acceleration Basics

CUDA Programming Model

CUDA (Compute Unified Device Architecture):

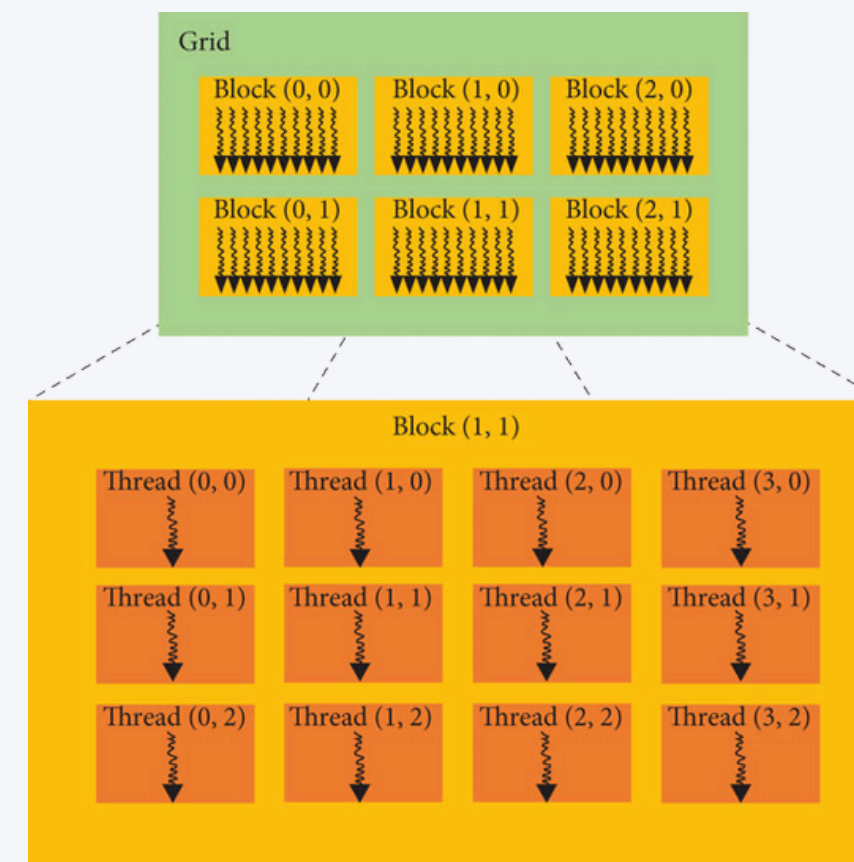
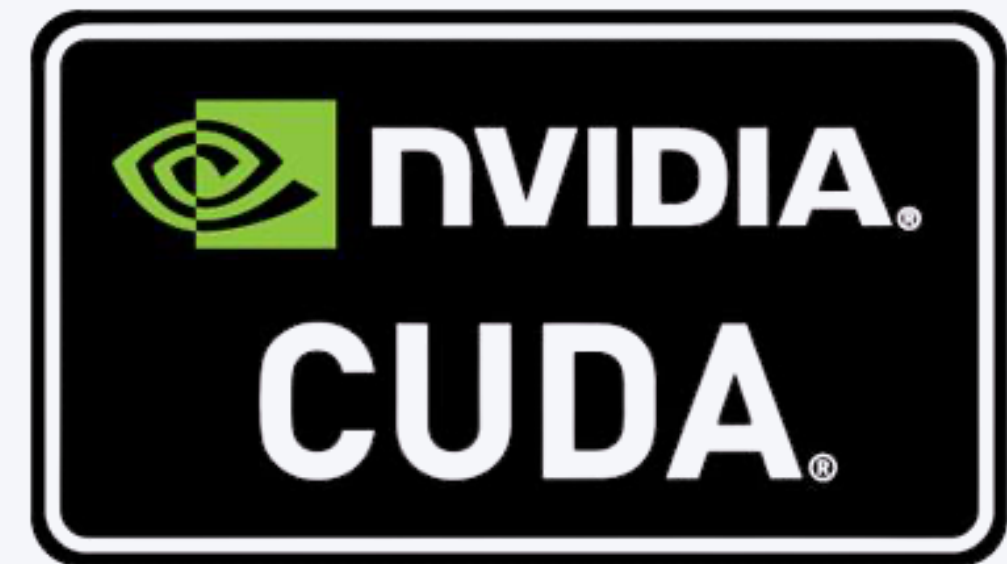
- Parallel computing platform and API by NVIDIA
- Allows developers to use GPUs for general-purpose processing
- C/C++ language extensions
- Provides both low-level and high-level APIs

Key Concepts:

- Host (CPU) and Device (GPU)
- Kernel functions
- Thread hierarchy
- Memory hierarchy

CUDA Thread Hierarchy:

- Thread: Individual execution unit
- Block: Group of threads that can communicate via shared memory
- Grid: Collection of blocks that execute the same kernel



Introduction To GPU Acceleration Basics

Why Use GPU Acceleration in Data Science?

- **Speed:** 10-100x faster for suitable algorithms
- **Scalability:** Process larger datasets
- **Energy Efficiency:** More computations per watt
- **Cost Effectiveness:** Cheaper than CPU clusters for many workloads
- **Real-time Analytics:** Enable interactive analysis of large datasets

Introduction To GPU Acceleration Basics

Common Algorithms Accelerated by GPUs

- **Machine Learning:**
- **Linear/Logistic Regression**
- **Gradient Boosting Machines (XGBoost, LightGBM)**
- **Neural Networks and Deep Learning**
- **Support Vector Machines**
- **K-Means Clustering**

Introduction To GPU Acceleration Basics

Python GPU Ecosystem Overview

- **Low-level:** CUDA Python, PyCUDA
- **Array computation:** CuPy, PyTorch, TensorFlow
- **Data processing:** RAPIDS (cuDF, cuML)
- **Specialized:** H2O4GPU, Numba, JAX
- **Distributed:** Dask-CUDA, RAPIDS + Dask

Introduction To GPU Acceleration Basics

Practical Considerations for GPU Computing

- **Entry-level (Development & Testing):**

- NVIDIA GTX/RTX Consumer Cards (e.g., RTX 3080, RTX 4090)
- 8-24 GB VRAM
- \$700-\$2,000

- **Professional (Data Science Workstations):**

- NVIDIA RTX A-series or previous Quadro series
- 16-48 GB VRAM
- \$2,000-\$6,000

- **High-Performance (Server/Cloud):**

- NVIDIA Tesla/A100/H100
- 40-80 GB VRAM
- \$8,000-\$30,000+



Hands-On Code

GPU Acceleration Libraries for Python

Activity: Checking Your Hardware



Open your laptops and follow these instructions based on your operating system:"

For Mac Users:

1. Click the Apple menu and select "About This Mac"
2. You'll see a summary of your CPU
3. Click "System Report..." then "Graphics/Displays" for GPU details

For Windows Users:

- Press **Win + X** and select "**Device Manager**"
- Expand "**Display adapters**" to see your GPU(s)
- Expand "**Processors**" to see your CPU

For more detailed information:

- Type "dxdiag" in the search bar and run it
- Check the "System" tab for CPU info and "Display" tab for GPU details